

Anthony Pham
Dr. Eman Ramadan
CSCI 4211
12 December 2025

Mininet Setup Experience Summary

Setting up Mininet was straightforward for me. I already had some experience with setting up a virtual machine on my computer. I also have used UTM before, so I was familiar with how it worked. This made following the tutorial a simple process. As I was following the VM setup tutorial, I did not run into any issues.

Walkthrough Tutorial Summary

The Mininet walkthrough was very helpful. Through reading it, I learned some useful tools and utilities for Mininet. An important one was the *ping* command. This allows me to see the average RTT between two nodes. Another one was the *ifconfig* command. This provides important information about the network/interfaces. Lastly, the *iperf* command lets me test the TCP bandwidth between two nodes.

Topology A Evaluation

Link	Host Pairs	<i>ping</i> Average RTT of 10 packets (ms)	<i>ping</i> Estimated Latency (avg. RTT/2)	<i>iperf</i> Average TCP Throughput (interval, overall)
L1	h1, h5	22.531	11.2655	28.5 Mbits/sec, 28.4 Mbits/sec
L2	h1, h2	83.982	41.991	18.4 Mbits/sec, 18.2 Mbits/sec
L3	h2, h3	65.794	32.897	36.6 Mbits/sec, 36.2 Mbits/sec
L4	h4, h7	43.987	21.9935	9.5 Mbits/sec, 9.4 Mbits/sec

Above is the data recorded from *ping* and *iperf*. For each *ping* call, I had ten packets sent and took the average RTT reported at the end. To find the *latency*, I divided the average RTT value by two. I chose to do this because RTT is the delay of *both* forward and return transmissions, so the latency would be half of that. However, this is assuming that RTT is symmetrical, meaning that the total delay is perfectly split between forward and backward transmissions. In reality, this is often not the case.

For throughput, *iperf* returns a set of measurements. These are the interval average and overall average, respectively. For each link, the interval average closely matches the overall

average. This tells me that the links have consistent throughput throughout the whole connection. In other words, the connections are very stable.

Topology B Evaluation

Link	Host Pairs	<i>ping</i> Raw Average RTT of 10 packets (ms)	<i>ping</i> Per-Link Average RTT of 10 packets (ms)	<i>ping</i> Estimated Latency (ms)	<i>iperf</i> Average TCP Throughput (interval, overall)
L3	h1, h2	44.061	22.0305	11.01525	9.35 Mbits/sec, 9.3 Mbits/sec
L2	h1, h3	154.598	55.2685	27.63425	17.4 Mbits/sec, 17.0 Mbits/sec
L1	h1, h8	341.931	71.636	35.818	15.3 Mbits/sec, 14.6 Mbits/sec

Measuring latency and throughput for topology B was a similar process to topology A's, except the RTT's in each layer builds off of the ones below it. In other words, the RTT of an upper layer link contains the RTT's of the links below it. Below is how I measured the latency for L2 and L1:

1. Record the raw average RTT value measured by *ping* between hosts.
2. Subtract the raw RTT value by the raw RTT values of the previous layers. Divide the result by two to get the per-link average RTT.
3. Divide the per-link average RTT by two to get the one-way latency. (This is assuming that the path is symmetrical)

Looking at the estimated latencies, it seems that L3 and L2's latencies are in line with the delays set in the Python program. L3's 11.01525ms latency is just above the 10ms delay set in the code, and L2's 27.63425ms latency is only slightly higher than its set 25ms delay. L3's is an interesting case, since its latency is actually *lower* than the set delay of 45ms by almost 10ms. This could signify an error in my calculation or a result of software overhead.

I believe that it is not possible to evaluate the throughput of the links between the switches, at least not accurately. This is because throughput is determined by the bottleneck within the links. The throughput is only as fast as the slowest link. This property makes throughput not additive like latency is, so you can not break down the measured throughput link-by-link.

Ethernet Learning Initial Implementation

- Pinging h1 from h2 was unsuccessful after I first ran the controller and topology C.
- Upon running *dpctl dump-flows*, it seems that the s1 switch has not received any rules yet.
- The cause of the pinging not working is likely due to the switch not having any rules yet. This makes sense, since the program has not been fully implemented yet.

```
mininet> h1 ping h2 -c 1
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
^C
--- 10.0.0.2 ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms
```

d.

```
mininet> dpctl dump-flow
*** s1 -----
```

Ethernet Learning Second Implementation

After the first round of implementation and pinging from h1's terminal, these are the results:

```
Node: h1
root@mininet:/home/mininet/Downloads# ping 10.0.0.2 -c 3
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.30 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.180 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.060 ms

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2029ms
rtt min/avg/max/mdev = 0.060/0.511/1.295/0.556 ms
root@mininet:/home/mininet/Downloads#
```

h1 pings three times, with an average RTT of 0.511. Time for each subsequent packet is significantly reduced.

```
root@mininet:/home/mininet/Downloads# tcpdump -n -i h2-eth0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on h2-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
00:12:00.242106 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 42729, seq 1, length 64
00:12:00.242122 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 42729, seq 1, length 64
00:12:01.242467 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 42729, seq 2, length 64
00:12:01.242481 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 42729, seq 2, length 64
00:12:02.270096 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 42729, seq 3, length 64
00:12:02.270109 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 42729, seq 3, length 64
00:13:04.057782 IP6 fe80::200:ff:fe00:2 > ff02::2: ICMP6, router solicitation, length 16
00:13:18.078903 IP6 fe80::6846:75ff:fe68:7891,5353 > ff02::fb,5353: 0 [10q] PTR (QM)? _nfs._tcp.local. PTR (QM)? _ipp._tcp.local. PTR (QM)? _ipps._tcp.local. PTR (QM)? _pgpkey-hkp._tcp.local. PTR (QM)? _ftp._tcp.local. PTR (QM)? _webdav._tcp.local. PTR (QM)? _webdavs._tcp.local. PTR (QM)? _sftp-ssh._tcp.local. PTR (QM)? _smb._tcp.local. PTR (QM)? _afpovertcp._tcp.local. (159)
00:13:35.829896 IP6 fe80::6846:75ff:fe68:7891 > ff02::2: ICMP6, router solicitation, length 16
```

h2 receives the three pings, replies to each of them.

```
root@mininet:/home/mininet/Downloads# tcpdump -n -i h3-eth0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on h3-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
00:12:35.385760 IP6 fe80::144e:a7ff:fe24:43b > ff02::2: ICMP6, router solicitation, length 16
00:13:16.345776 IP6 fe80::200:ff:fe00:3 > ff02::2: ICMP6, router solicitation, length 16
00:13:18.113090 IP6 fe80::144e:a7ff:fe24:43b,5353 > ff02::fb,5353: 0 [10q] PTR (QM)? _nfs._tcp.local. PTR (QM)? _ipp._tcp.local. PTR (QM)? _ipps._tcp.local. PTR (QM)? _pgpkey-hkp._tcp.local. PTR (QM)? _ftp._tcp.local. PTR (QM)? _webdav._tcp.local. PTR (QM)? _webdavs._tcp.local. PTR (QM)? _sftp-ssh._tcp.local. PTR (QM)? _smb._tcp.local. PTR (QM)? _afpovertcp._tcp.local. (159)
00:16:20.666346 IP6 fe80::144e:a7ff:fe24:43b > ff02::2: ICMP6, router solicitation, length 16
```

h3 is unaffected, remains listening.

After the first ping, the latency per packet is significantly reduced. Now that the controller algorithm is filled in and addresses are remembered, the source MAC-port mapping is stored upon the first ping and used for the following pings.

Running iPerf between h1 and h2 seems to work. The console logs do not indicate much of what's going on behind the scenes, so that tells me that the controller never received any of the packets since the mapping has already been done from the pings.

After starting clean and rerunning iPerf, there seems to be 8 rules installed in S1 (in_port, dl_src, dl_dst, nw_src, nw_dst, nw_tos, tp_src, tp_dst). These must have been matched and installed when the first few packets had gone through the controller.

"Node: h2"

```
root@mininet:/home/mininet/Downloads# iperf -s
```

```
-----  
Server listening on TCP port 5001  
TCP window size: 85.3 KByte (default)  
-----
```

```
[  1] local 10.0.0.2 port 5001 connected with 10.0.0.1 port 35226  
[ ID] Interval      Transfer    Bandwidth  
[  1] 0.0000-9.9561 sec 92.9 GBytes 80.2 Gbits/sec
```

"Node: h1"

```
root@mininet:/home/mininet/Downloads# iperf -c 10.0.0.2
```

```
-----  
Client connecting to 10.0.0.2, TCP port 5001  
TCP window size: 85.3 KByte (default)  
-----
```

```
[  1] local 10.0.0.1 port 35226 connected with 10.0.0.2 port 5001  
[ ID] Interval      Transfer    Bandwidth  
[  1] 0.0000-10.0057 sec 92.9 GBytes 79.8 Gbits/sec
```

```
root@mininet:/home/mininet/Downloads#
```

Ethernet Learning Third Implementation

- a. iPerf worked this round
- b. iPerf packets were likely not sent to the controller
- c. 2 rules
- d. I assumed that no packets were sent to the controller because iPerf ran right after Ping. The MAC-port mappings were already done during the pinging.

When I ran *sh ovs-ofctl dump-flows s1* in the Mininet command line, I saw only the `dl_src` and `dl_dst` rules. This is expected, since the new code

only matches on the source and destination MAC addresses.

```
"Node: h2"
01:07:36.870945 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 9073, seq 3, length 64
01:08:38.203045 IP6 fe80::200:ff:fe00:2 > ff02::2: ICMP6, router solicitation, length 16
01:08:42.300480 IP6 fe80::a6:60ff:fe4b:e8b5 > ff02::2: ICMP6, router solicitation, length 16
01:08:53.413154 IP6 fe80::a6:60ff:fe4b:e8b5,5353 > ff02::fb,5353: 0 [10q] PTR (QM)? _nfs._tcp.local. PTR (QM)? _ipp._tcp.local. PTR (QM)? _ipps._tcp.local. PTR (QM)? _pgpkey-hkp._tcp.local. PTR (QM)? _ftp._tcp.local. PTR (QM)? _webdav._tcp.local. PTR (QM)? _webdavs._tcp.local. PTR (QM)? _sftp-ssh._tcp.local. PTR (QM)? _smb._tcp.local. PTR (QM)? _afpovertcp._tcp.local. (159)
^[[A^C
9 packets captured
9 packets received by filter
0 packets dropped by kernel
root@mininet:/home/mininet/Downloads# iperf -s -p 4000
-----
Server listening on TCP port 4000
TCP window size: 85.3 KByte (default)
-----
[  1] local 10.0.0.2 port 4000 connected with 10.0.0.1 port 42574
[ ID] Interval      Transfer    Bandwidth
[  1] 0.0000-9.9998 sec  109 GBytes  93.3 Gbits/sec
█

"Node: h1"
root@mininet:/home/mininet/Downloads# ping 100.0.2 -c 3
ping: connect: Network is unreachable
root@mininet:/home/mininet/Downloads# ping 10.0.0.2 -c 3
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=51.4 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.878 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.080 ms

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.080/17.460/51.424/24.017 ms
root@mininet:/home/mininet/Downloads# iperf -c 10.0.0.2 -p 4000
-----
Client connecting to 10.0.0.2, TCP port 4000
TCP window size: 340 KByte (default)
-----
[  1] local 10.0.0.1 port 42574 connected with 10.0.0.2 port 4000
[ ID] Interval      Transfer    Bandwidth
[  1] 0.0000-10.0137 sec  109 GBytes  93.2 Gbits/sec
root@mininet:/home/mininet/Downloads# █
```


pingall Evaluation

- a. i. 5 packets were sent to the controller
- ii. 4 messages were flooded

```
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
iii. mininet> 
```

```
mininet> dpctl dump-flows
*** s1 -----
cookie=0x0, duration=374.588s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:02, dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
cookie=0x0, duration=374.544s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:01, dl_dst=00:00:00:00:00:02 actions=output:"s1-eth2"
cookie=0x0, duration=374.539s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:03, dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
cookie=0x0, duration=374.496s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:01, dl_dst=00:00:00:00:00:03 actions=output:"s1-eth3"
cookie=0x0, duration=374.492s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:02, dl_dst=00:00:00:00:00:03 actions=output:"s1-eth3"
cookie=0x0, duration=374.446s, table=0, n_packets=2, n_bytes=196, dl_src=00:00:00:00:00:03, dl_dst=00:00:00:00:00:02 actions=output:"s1-eth2"
iv. mininet> 
```

- b. The controller seemed to not receive any packets during the second pingall, as the console did not report any packages arriving. This is likely because all the mappings had been done in the first round.

Topology A and B Re-Evaluation

Topology	Packets Sent to Controller	Messages Flooded by Controller
A	171	130
B	195	233

Rules installed in s1

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8 h9
h2 -> h1 h3 h4 h5 h6 h7 h8 h9
h3 -> h1 h2 h4 h5 h6 h7 h8 h9
h4 -> h1 h2 h3 h5 h6 h7 h8 h9
h5 -> h1 h2 h3 h4 h6 h7 h8 h9
h6 -> h1 h2 h3 h4 h5 h7 h8 h9
h7 -> h1 h2 h3 h4 h5 h6 h8 h9
h8 -> h1 h2 h3 h4 h5 h6 h7 h9
h9 -> h1 h2 h3 h4 h5 h6 h7 h8
*** Results: 0% dropped (72/72 received)
mininet>
```

Topology A's reachable hosts

```
** c1 -----
cookie=0x0, duration=125.820s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:05,dl_dst=00:00:00:00:00:01 actions=output:"c1-eth1"
cookie=0x0, duration=125.820s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:05 actions=output:"c1-eth2"
cookie=0x0, duration=125.436s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:06,dl_dst=00:00:00:00:00:01 actions=output:"c1-eth1"
cookie=0x0, duration=125.436s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:06 actions=output:"c1-eth2"
cookie=0x0, duration=125.038s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:07,dl_dst=00:00:00:00:00:01 actions=output:"c1-eth1"
cookie=0x0, duration=125.038s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:07 actions=output:"c1-eth2"
cookie=0x0, duration=124.650s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:08,dl_dst=00:00:00:00:00:01 actions=output:"c1-eth1"
cookie=0x0, duration=124.650s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:08 actions=output:"c1-eth2"
cookie=0x0, duration=124.119s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:05 actions=output:"c1-eth2"
cookie=0x0, duration=124.119s, table=0, n_packets=2, n_bytes=196, dl_src=00:00:00:00:00:05,dl_dst=00:00:00:00:00:02 actions=output:"c1-eth1"
cookie=0x0, duration=123.772s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:06 actions=output:"c1-eth2"
cookie=0x0, duration=123.772s, table=0, n_packets=2, n_bytes=196, dl_src=00:00:00:00:00:06,dl_dst=00:00:00:00:00:02 actions=output:"c1-eth1"
cookie=0x0, duration=123.412s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:07 actions=output:"c1-eth2"
cookie=0x0, duration=123.412s, table=0, n_packets=2, n_bytes=196, dl_src=00:00:00:00:00:07,dl_dst=00:00:00:00:00:02 actions=output:"c1-eth1"
cookie=0x0, duration=123.050s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:08 actions=output:"c1-eth2"
cookie=0x0, duration=123.050s, table=0, n_packets=2, n_bytes=196, dl_src=00:00:00:00:00:08,dl_dst=00:00:00:00:00:02 actions=output:"c1-eth1"
cookie=0x0, duration=122.321s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:05 actions=output:"c1-eth2"
cookie=0x0, duration=122.321s, table=0, n_packets=2, n_bytes=196, dl_src=00:00:00:00:00:05,dl_dst=00:00:00:00:00:03 actions=output:"c1-eth1"
cookie=0x0, duration=121.969s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:06 actions=output:"c1-eth2"
cookie=0x0, duration=121.969s, table=0, n_packets=2, n_bytes=196, dl_src=00:00:00:00:00:06,dl_dst=00:00:00:00:00:03 actions=output:"c1-eth1"
cookie=0x0, duration=121.634s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:07 actions=output:"c1-eth2"
cookie=0x0, duration=121.634s, table=0, n_packets=2, n_bytes=196, dl_src=00:00:00:00:00:07,dl_dst=00:00:00:00:00:03 actions=output:"c1-eth1"
cookie=0x0, duration=121.287s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:08 actions=output:"c1-eth2"
cookie=0x0, duration=121.287s, table=0, n_packets=2, n_bytes=196, dl_src=00:00:00:00:00:08,dl_dst=00:00:00:00:00:03 actions=output:"c1-eth1"
cookie=0x0, duration=120.587s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:05 actions=output:"c1-eth2"
cookie=0x0, duration=120.587s, table=0, n_packets=2, n_bytes=196, dl_src=00:00:00:00:00:05,dl_dst=00:00:00:00:00:04 actions=output:"c1-eth1"
cookie=0x0, duration=120.245s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:06 actions=output:"c1-eth2"
cookie=0x0, duration=120.245s, table=0, n_packets=2, n_bytes=196, dl_src=00:00:00:00:00:06,dl_dst=00:00:00:00:00:04 actions=output:"c1-eth1"
cookie=0x0, duration=119.876s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:07 actions=output:"c1-eth2"
cookie=0x0, duration=119.876s, table=0, n_packets=2, n_bytes=196, dl_src=00:00:00:00:00:07,dl_dst=00:00:00:00:00:04 actions=output:"c1-eth1"
cookie=0x0, duration=119.534s, table=0, n_packets=1, n_bytes=98, dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:08 actions=output:"c1-eth2"
cookie=0x0, duration=119.534s, table=0, n_packets=2, n_bytes=196, dl_src=00:00:00:00:00:08,dl_dst=00:00:00:00:00:04 actions=output:"c1-eth1"
```

Rules installed in c1

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8
h2 -> h1 h3 h4 h5 h6 h7 h8
h3 -> h1 h2 h4 h5 h6 h7 h8
h4 -> h1 h2 h3 h5 h6 h7 h8
h5 -> h1 h2 h3 h4 h6 h7 h8
h6 -> h1 h2 h3 h4 h5 h7 h8
h7 -> h1 h2 h3 h4 h5 h6 h8
h8 -> h1 h2 h3 h4 h5 h6 h7
*** Results: 0% dropped (56/56 received)
mininet>
```

Topology B's reachable hosts

Topology A:

Link	Host Pairs	Previous <i>ping</i> Estimated Latency (avg. RTT/2)	New <i>ping</i> Estimated Latency	Previous <i>iperf</i> Average TCP Throughput (<i>interval, overall</i>)	New <i>iperf</i> Average TCP Throughput
L1	h1, h5	11.2655	13.7525	28.5 Mbits/sec, 28.4 Mbits/sec	28.5 Mbits/sec, 28.4 Mbits/sec
L2	h1, h2	41.991	45.933	18.4 Mbits/sec, 18.2 Mbits/sec	18.4 Mbits/sec, 18.1 Mbits/sec
L3	h2, h3	32.897	34.187	36.6 Mbits/sec, 36.2 Mbits/sec	36.9 Mbits/sec, 36.5 Mbits/sec
L4	h4, h7	21.9935	23.952	9.5 Mbits/sec, 9.4 Mbits/sec	9.4 Mbits/sec, 9.4 Mbits/sec

The new pings in topology A took slightly longer than the previous round. This makes sense, since there may be some additional overhead with my controller and may not be as optimized as the default one. Throughputs were more or less the same. This may be because the rule setting done in my controller only adds to the time it takes to process packets, not affecting the bandwidth in any way.

Topology B:

Link	Host Pairs	Previous <i>ping</i> Estimated Latency (<i>ms</i>)	New <i>ping</i> Estimated Latency	Previous <i>iperf</i> Average TCP Throughput (<i>interval, overall</i>)	New <i>iperf</i> Average TCP Throughput
L3	h1, h2	11.01525	12.32925 (49.317 RTT)	9.35 Mbits/sec, 9.3 Mbits/sec	18.9 Mbits/sec, 18.7 Mbits/sec
L2	h1, h3	27.63425	27.25825 (158.350 RTT)	17.4 Mbits/sec, 17.0 Mbits/sec	17.5 Mbits/sec, 17.1 Mbits/sec
L1	h1, h8	35.818	35.16925 (348.344 RTT)	15.3 Mbits/sec, 14.6 Mbits/sec	15.1 Mbits/sec, 14.4 Mbits/sec

For topology B, the new latencies are closer to the previous latencies compared to topology A's. The reasoning behind this may have to do with the structure of topology B, since the nodes are structured in layers. This means that there are more common nodes between hosts,

so there is less mapping needed to be done. The throughputs are also very similar to the previous values, though L3's is double the previous values. I do not have an explanation for this.